ENGINEERING TRAINING · SESSION HANDBOOK

Design Patterns & the Repository Pattern

A practical reference for .NET developers — the three pattern categories, the Repository Pattern end-to-end, a generic repository with Unit of Work in ASP.NET Core and EF Core, and the judgement to know when not to use it at all.

BY THE END OF THIS SESSION YOU WILL BE ABLE TO

1. Recognise the three design pattern categories: creational, structural, behavioural.
2. Explain the Repository Pattern's intent and structure.
3. Implement a generic repository with a Unit of Work pattern.
4. Identify when the pattern adds value and when it adds unnecessary layers.

Prepared by Yazan Faroun
Trainee reference · read before the session, keep open during the lab

PREPARED USING CLAUDE

C# 14 · .NET 10 · EF Core 10
Examples compile against ASP.NET Core

BEFORE YOU START

How to use this handbook

BEFORE THE SESSION

Read Part 1 and §2.1–2.4. Twenty minutes. You will get far more out of the live walkthrough if the vocabulary is already familiar.

DURING THE SESSION

Keep Part 3 open — we build exactly that code together, in that order. Figures 3.1 and 3.2 are the two slides worth annotating.

AFTER THE SESSION

Run the lab in Appendix A against your own project, then use the §4.6 checklist on the next pull request you review.

Every code sample targets **C# 14 on .NET 10 with EF Core 10** and uses primary constructors, collection expressions and file-scoped namespaces. Samples are written to be pasted and adapted, not admired: names come from a claims/billing domain, but nothing in the pattern depends on it.

Three conventions are used throughout. A `RED-TOPPED` listing shows code you should not ship; a `GREEN-TOPPED` listing shows the improvement; and every part closes with a four-line summary you can revise from.

Contents

Part 1 · Design Patterns — The Landscape 3

1.1 What a design pattern actually is 3
1.2 Why the categories exist 3
1.3 The catalogue at a glance — all 23 GoF patterns 4
1.4 Creational patterns in code 5
1.5 Structural patterns in code 8
1.6 Behavioural patterns in code 10
1.7 Telling the categories apart — a working rubric 12

Part 2 · The Repository Pattern — Intent & Structure 13

2.1 The problem it was invented for 13
2.2 Intent 14
2.3 Structure and participants 14
2.4 The same query, mediated 15
2.5 Where the repository sits in a layered architecture 16
2.6 Patterns it is confused with 16
2.7 The honest objection: “EF Core is already a repository” 17
2.8 The leak that decides the design 17

Part 3 · Building a Generic Repository with Unit of Work 19

3.1 What we are building 19
3.2 Step 1 — entity base and DbContext 19
3.3 Step 2 — the repository contract 20
3.4 Step 3 — the Specification pattern 21
3.5 Step 4 — the generic repository implementation 22
3.6 Step 5 — the Unit of Work 24
3.7 Step 6 — the sequence of one request 26
3.8 Step 7 — registration and lifetimes 26
3.9 Step 8 — using it, and aggregate-specific repositories 27
3.10 Step 9 — concurrency 29
3.11 Step 10 — testing 29
3.12 Bugs this design is built to prevent 31

Part 4 · Judgement — Value vs. Unnecessary Layers 32

4.1 What the pattern actually buys you 32
4.2 What it does not buy you 33
4.3 The anti-pattern gallery 33
4.4 The decision procedure 34
4.5 Comparing the realistic options 34
4.6 A checklist you can run in a pull request 35
4.7 Getting from here to there 36

Appendices 37

A Lab exercises — classify, build, argue 37
B Quick reference, glossary and further reading 39

Figures and tables

Fig 1.1 The three categories
Fig 1.2 Factory Method structure
Fig 1.3 A decorator chain
Fig 1.4 Strategy structure
Fig 2.1 Repository participants
Fig 2.2 Placement in a layered architecture
Fig 3.1 Generic repository + Unit of Work
Fig 3.2 One request, two aggregates, one commit
Fig 4.1 When to introduce the layer

Tbl 1.1 The Gang of Four catalogue (all 23)
Tbl 2.1 What unmediated data access costs
Tbl 2.2 Participants and responsibilities
Tbl 2.3 Neighbouring patterns compared
Tbl 3.1 DI lifetimes and what breaks
Tbl 3.2 Failure modes and their guards
Tbl 4.1 Benefits and their proof
Tbl 4.2 Five data-access approaches scored

PART ONE

1 Design Patterns — The Landscape

> **Objective 1.** Recognise the three design pattern categories: creational, structural, behavioural — and know which problems each category is named after.

1.1 What a design pattern actually is

A design pattern is a **named, reusable solution to a problem that keeps recurring in object-oriented design**. It is not a library, not a framework, and not code you copy. It is a description of how a small set of classes and objects can be arranged so that a particular kind of change becomes cheap.

The term was borrowed from the architect Christopher Alexander, who catalogued recurring arrangements in buildings and towns. In 1994 Erich Gamma, Richard Helm, Ralph Johnson and John Vlissides — collectively the *Gang of Four* (GoF) — published *Design Patterns: Elements of Reusable Object-Oriented Software*, which documented 23 such arrangements for software. Those 23 are still the shared vocabulary of the industry, which is the single biggest reason to learn them.

A PATTERN IS

- A **name** the whole team recognises
- A **problem** statement and its context
- A **structure** of collaborating types
- A set of **consequences** — what you gain, what you pay

A PATTERN IS NOT

- A goal in itself — "using more patterns" is not better design
- Language-independent gospel — many GoF patterns are a `Func<T>` or an interface in C#
- An excuse for layers nobody asked for
- A substitute for understanding the domain

> **THE ONE-SENTENCE DEFINITION**
> A design pattern is **a named trade-off**: you accept extra indirection today so that a specific, expected kind of change is cheap tomorrow. If that change never comes, you paid for nothing.

1.2 Why the categories exist

The GoF sorted their catalogue along one axis that turns out to be genuinely useful: *what aspect of the system is the pattern trying to make flexible?* Answering that question gives you three buckets.

CATEGORY 01

Creational

How objects get made

"new is a commitment."



Decouples a caller from the concrete class it instantiates.

Factory · Builder · Singleton

CATEGORY 02

Structural

How objects are composed

"make these two fit."



Combines types into larger structures without editing them.

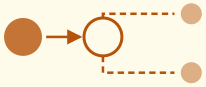
Adapter · Decorator · Facade

CATEGORY 03

Behavioural

How objects talk and decide

"who does what, when."



Assigns responsibility and controls the flow of messages.

Strategy · Observer · Command

Figure 1.1 — The three categories. Ask "what is this pattern making flexible?" Object *creation* → creational. Object *composition* → structural. Object *communication* → behavioural. Every GoF pattern lands in exactly one bucket.

HOW TO CLASSIFY A PATTERN YOU HAVE NEVER SEEN

Read its intent sentence and look for the verb. *Create, instantiate, build, clone* → creational. *Compose, wrap, adapt, expose, share* → structural. *Encapsulate a request, notify, iterate, vary an algorithm* → behavioural.

1.3 The catalogue at a glance

You are not expected to memorise all 23. You are expected to recognise the name when a colleague says it in a code review, and to know where to look it up. The eight marked `CORE` are the ones you will actually meet in a .NET codebase most weeks.

Pattern	Category	Intent in one line	Where you have already seen it in .NET
Abstract Factory	Creational	Create families of related objects without naming their concrete classes.	`DbProviderFactory`
Builder `CORE`	Creational	Assemble a complex object step by step; same steps, different results.	`WebApplicationBuilder`, `StringBuilder`
Factory Method `CORE`	Creational	Let a subclass or delegate decide which concrete type to instantiate.	`ILoggerFactory.CreateLogger`
Prototype	Creational	Create new objects by copying an existing configured instance.	`ICloneable`, record `with`
Singleton `CORE`	Creational	Guarantee one instance and a global access point to it.	DI `AddSingleton` (the safe version)
Adapter `CORE`	Structural	Convert one interface into another the client expects.	`TextReader` wrappers, SDK shims
Bridge	Structural	Split an abstraction from its implementation so both vary independently.	`ILogger` vs logging providers

Pattern	Category	Intent in one line	Where you have already seen it in .NET
Composite	Structural	Treat a tree of objects and a single object uniformly.	Razor component trees, `IConfiguration`
Decorator CORE	Structural	Add behaviour by wrapping, without touching the wrapped class.	`DelegatingHandler`, middleware
Facade CORE	Structural	Offer one simple entry point over a complicated subsystem.	`HttpClient` over sockets/TLS
Flyweight	Structural	Share immutable state across many objects to save memory.	String interning, `ArrayPool<T>`
Proxy	Structural	Stand in for another object to control access to it.	EF Core lazy-loading proxies
Chain of Responsibility	Behavioural	Pass a request along a chain until someone handles it.	ASP.NET Core middleware pipeline
Command CORE	Behavioural	Wrap a request as an object so it can be queued, logged or undone.	MediatR requests, `ICommand` in WPF
Interpreter	Behavioural	Represent a grammar and evaluate sentences in it.	LINQ expression trees
Iterator	Behavioural	Walk a collection without exposing its internals.	`IEnumerable<T>` / `yield return`
Mediator	Behavioural	Route communication through one object instead of many direct links.	MediatR
Memento	Behavioural	Capture and restore an object's state without breaking encapsulation.	EF Core change tracker snapshots
Observer CORE	Behavioural	Notify many dependents automatically when one object changes.	`IObservable<T>`, events, RxJS
State	Behavioural	Change an object's behaviour when its internal state changes.	Workflow / status machines
Strategy CORE	Behavioural	Make a family of algorithms interchangeable at run time.	`IComparer<T>`, pricing rules
Template Method	Behavioural	Fix the skeleton of an algorithm, let subclasses fill in steps.	`ControllerBase` filters, `BackgroundService`
Visitor	Behavioural	Add an operation to a class hierarchy without editing the classes.	`ExpressionVisitor`

Table 1.1 — The Gang of Four catalogue. Twenty-three patterns, three categories. The "where you have seen it" column matters more than the definition: patterns are easier to learn as recognition than as recipes.

1.4 Creational patterns in code

Creational patterns exist because the keyword `new` hard-codes a decision. The moment a class writes `new SqlEmailSender()`, that class can never be used with a different sender — not in a test, not in a

second deployment, not on a customer who needs SMTP.

Factory Method

Intent: define an interface for creating an object, but let the implementation decide which class to instantiate.

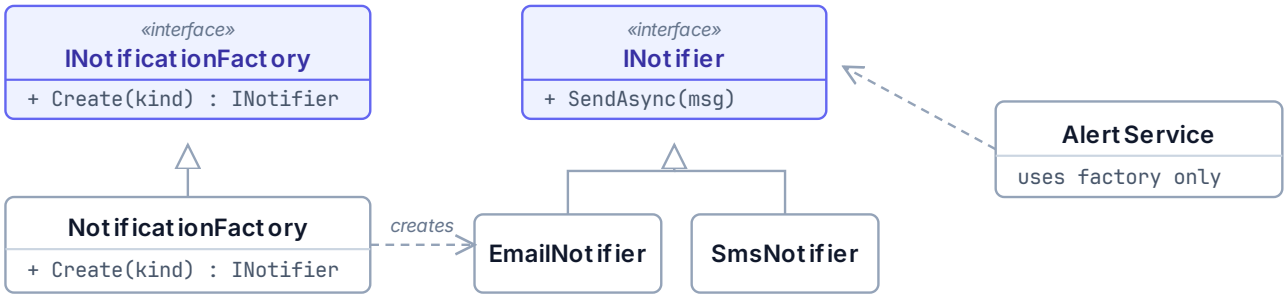


Figure 1.2 — Factory Method. `AlertService` depends on `INotificationFactory` and `INotifier` only. Adding a `PushNotifier` touches the factory and one new class — never the service.

BEFORE — THE DECISION IS WELDED INTO THE CALLER — AlertService.cs

```
public class AlertService
{
    public async Task RaiseAsync(Alert alert)
    {
        // Every new channel means editing this class. Untestable without a real SMTP server.
        if (alert.Channel == "sms")
            await new SmsNotifier("account-sid", "token").SendAsync(alert.Text);
        else
            await new EmailNotifier("smtp.internal", 587).SendAsync(alert.Text);
    }
}
```

AFTER — FACTORY METHOD — Notifications/*.cs

```
public interface INotifier
{
    Task SendAsync(string message, CancellationToken ct = default);
}

public interface INotificationFactory
{
    INotifier Create(NotificationChannel channel);
}

public sealed class NotificationFactory(IServiceProvider services) : INotificationFactory
{
    public INotifier Create(NotificationChannel channel) => channel switch
    {
        NotificationChannel.Sms   => services.GetRequiredService<SmsNotifier>(),
        NotificationChannel.Email => services.GetRequiredService<EmailNotifier>(),
        NotificationChannel.Push  => services.GetRequiredService<PushNotifier>(),
        _ => throw new ArgumentOutOfRangeException(nameof(channel), channel, "Unknown channel.")
    };
}

public sealed class AlertService(INotificationFactory factory)
```

```
{
    public Task RaiseAsync(Alert alert, CancellationToken ct = default) =>
        factory.Create(alert.Channel).SendAsync(alert.Text, ct);
}
```

C# REALITY CHECK

In modern .NET the DI container *is* an abstract factory. Before writing a factory class, ask whether injecting `IEnumerable<INotifier>`, a keyed service (`[FromKeyedServices("sms")]`), or a plain `Func<Channel, INotifier>` would do the same job with one less type.

Builder

Intent: separate the construction of a complex object from its representation, so the same construction process can produce different results. You use this every time you start an ASP.NET Core app.

BUILDER — A FLUENT, VALIDATED CONSTRUCTION PATH — ClaimBuilder.cs

```
public sealed class ClaimBuilder
{
    private readonly List<ClaimLine> _lines = [];
    private Guid _patientId;
    private DateOnly _serviceDate = DateOnly.FromDateTime(DateTime.UtcNow);
    private string? _payerCode;

    public ClaimBuilder ForPatient(Guid patientId)      { _patientId = patientId;   return this; }
    public ClaimBuilder OnDate(DateOnly date)           { _serviceDate = date;      return this; }
    public ClaimBuilder BilledTo(string payerCode)      { _payerCode = payerCode;   return this; }

    public ClaimBuilder AddLine(string code, int qty, decimal unitPrice)
    {
        _lines.Add(new ClaimLine(code, qty, unitPrice));
        return this;
    }

    public Claim Build()
    {
        if (_patientId == Guid.Empty)      throw new InvalidOperationException("Patient is required.");
        if (string.IsNullOrWhiteSpace(_payerCode)) throw new InvalidOperationException("Payer is
required.");
        if (_lines.Count == 0)             throw new InvalidOperationException("A claim needs at least one
line.");

        return new Claim(_patientId, _serviceDate, _payerCode, _lines.AsReadOnly());
    }
}

// usage — reads like the domain, and cannot produce a half-built Claim
var claim = new ClaimBuilder()
    .ForPatient(patientId)
    .BilledTo("PBM-01")
    .AddLine("A4253", qty: 100, unitPrice: 0.42m)
    .AddLine("E0607", qty: 1,   unitPrice: 38.00m)
    .Build();
```

Singleton — and why you should almost never hand-roll it

Intent: ensure a class has exactly one instance. The classic implementation is also the classic source of untestable, globally-coupled code, because it combines two unrelated responsibilities: *being unique* and *being reachable from anywhere*.

HAND-ROLLED — AVOID

```
public sealed class Cache
{
    private static readonly Lazy<Cache> _i =
        new(() => new Cache());

    public static Cache Instance => _i.Value;
    private Cache() { }
}

// caller is now welded to Cache
Cache.Instance.Get("k");
```

CONTAINER-MANAGED — PREFER

```
public interface ICache { T? Get<T>(string key); }

builder.Services.AddSingleton<ICache, MemoryCache>
();

// caller depends on an abstraction and
// can be given a fake in tests
public sealed class PriceService(ICache cache)
{
    /* ... */
}
```

CAPTIVE DEPENDENCY

A singleton that takes a scoped dependency in its constructor **captures** it for the lifetime of the application. Injecting a `DbContext` (or a repository, or a unit of work) into a singleton is one of the most common production bugs in ASP.NET Core. Inject `IServiceScopeFactory` and create a scope per operation instead.

1.5 Structural patterns in code

Structural patterns answer: *I have these types; how do I combine them without rewriting them?* Three carry almost all the weight in .NET.

Adapter

Intent: make an existing class usable through the interface your code already expects. This is the pattern you reach for whenever a third-party SDK does not match your abstraction — which is most of the time.

ADAPTER — A VENDOR SDK BEHIND YOUR OWN INTERFACE — TwilioSmsAdapter.cs

```
// Your application's contract — owned by you, shaped by your domain.
public interface INotifier
{
    Task SendAsync(string message, CancellationToken ct = default);
}

// The vendor's type — owned by someone else, shaped by their product.
// public sealed class TwilioRestClient { public Task<MessageResource> CreateMessageAsync(...); }

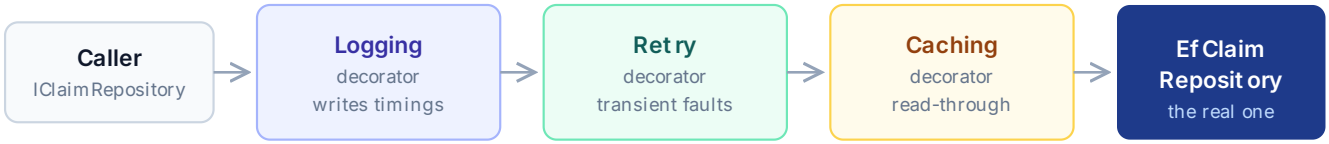
public sealed class TwilioSmsAdapter(TwilioRestClient client, IOptions<SmsOptions> options)
    : INotifier
{
    private readonly SmsOptions _o = options.Value;

    public async Task SendAsync(string message, CancellationToken ct = default)
    {
        // Translate our vocabulary into theirs, and their failures into ours.
        try
        {
            await client.CreateMessageAsync(
                to: _o.DefaultRecipient, from: _o.SenderId, body: message, cancellationToken: ct);
        }
        catch (ApiException ex)
        {
            throw new NotificationFailedException("SMS delivery failed.", ex);
        }
```

```
    }
}
```

Decorator

Intent: attach responsibilities to an object dynamically by wrapping it in another object with the same interface. Decorators compose: caching around retry around logging around the real thing, each written once and unaware of the others.



every box implements the same interface — the caller cannot tell how deep the stack is

Figure 1.3 — A decorator chain. Each wrapper implements `IClaimRepository` and holds an inner `IClaimRepository`. Order is configuration, not code: registering retry inside caching gives different semantics from caching inside retry.

DECORATOR — CACHING WITHOUT TOUCHING THE REAL REPOSITORY — CachingClaimRepository.cs

```
public sealed class CachingClaimRepository(
    IClaimRepository inner,
    IMemoryCache cache,
    ILogger<CachingClaimRepository> logger) : IClaimRepository
{
    private static readonly TimeSpan Ttl = TimeSpan.FromMinutes(5);

    public async Task<Claim?> GetByIdAsync(Guid id, CancellationToken ct = default)
    {
        if (cache.TryGetValue($"claim:{id}", out Claim? cached))
        {
            logger.LogDebug("Cache hit for claim {ClaimId}", id);
            return cached;
        }

        var claim = await inner.GetByIdAsync(id, ct);
        if (claim is not null)
            cache.Set($"claim:{id}", claim, Ttl);

        return claim;
    }

    // Writes must invalidate, or the decorator becomes a correctness bug.
    public async Task AddAsync(Claim claim, CancellationToken ct = default)
    {
        await inner.AddAsync(claim, ct);
        cache.Remove($"claim:{claim.Id}");
    }
}

// Registration — order reads outside-in.
builder.Services.AddScoped<IClaimRepository, EfClaimRepository>();
builder.Services.Decorate<IClaimRepository, CachingClaimRepository>();  // Scrutor
builder.Services.Decorate<IClaimRepository, LoggingClaimRepository>();
```

Facade

Intent: provide a single, simplified interface to a set of interfaces in a subsystem. A facade does not hide the subsystem — advanced callers can still go underneath — it just makes the common case one call instead of nine.

FACADE — ONE CALL OVER FOUR COLLABORATORS — ClaimSubmissionFacade.cs

```
public sealed class ClaimSubmissionFacade(
    IEligibilityChecker eligibility,
    IPricingEngine pricing,
    IClearingHouseClient clearingHouse,
    IAuditTrail audit)
{
    public async Task<SubmissionResult> SubmitAsync(Claim claim, CancellationToken ct = default)
    {
        var check = await eligibility.VerifyAsync(claim.PatientId, claim.ServiceDate, ct);
        if (!check.IsEligible)
            return SubmissionResult.Rejected(check.Reason);

        var priced   = await pricing.PriceAsync(claim, check.PlanId, ct);
        var receipt  = await clearingHouse.SendAsync(priced, ct);
        await audit.RecordAsync(claim.Id, receipt.TrackingNumber, ct);

        return SubmissionResult.Accepted(receipt.TrackingNumber);
    }
}
```

FACADE VS. GOD CLASS

A facade *delegates*; it holds no business rules of its own. The moment your facade starts computing prices rather than asking the pricing engine, it has stopped being a facade and become the class everyone is afraid to edit.

1.6 Behavioural patterns in code

Strategy

Intent: define a family of algorithms, encapsulate each one, and make them interchangeable. Strategy is the cure for the long `switch` that grows a new arm every sprint.

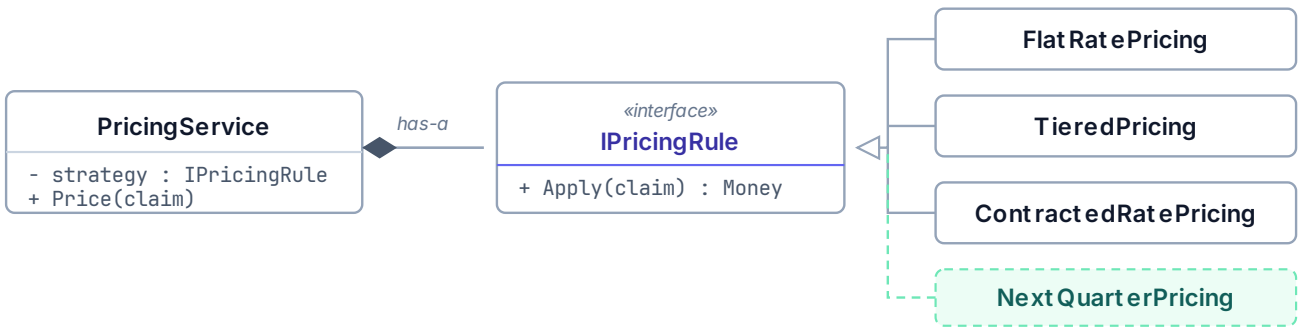


Figure 1.4 — Strategy. The dashed box is the point of the pattern: a new algorithm is a new class, not an edit to `PricingService`. That is the Open/Closed Principle made concrete.

BEFORE — THE SWITCH THAT NEVER STOPS GROWING

```
public decimal Price(Claim c) => c.PlanType switch
{
    "flat"       => c.Lines.Sum(l => l.Qty * 10m),
    "tiered"     => TieredMath(c),
    "contracted" => LookupContract(c),
    // + one more arm per sprint,
    //   forever, in one file
    _ => throw new NotSupportedException()
};
```

AFTER — STRATEGY SELECTED BY THE CONTAINER

```
public interface IPricingRule
{
    string PlanType { get; }
    Money Apply(Claim claim);
}

public sealed class PricingService(
    IEnumerable<IPricingRule> rules)
{
    public Money Price(Claim claim) =>
        rules.SingleOrDefault(r => r.PlanType ==
claim.PlanType)
            ?.Apply(claim)
        ?? throw new NotSupportedException(
               $"No pricing rule for
'{claim.PlanType}'.");
}
```

Observer

Intent: define a one-to-many dependency so that when one object changes state, all its dependents are notified. In C# this is `event`, `IObservable<T>` or a domain-event dispatcher; in Angular it is `Observable` and `Subject` — the same pattern, both ends of your stack.

OBSERVER — DOMAIN EVENTS RAISED BY THE AGGREGATE, DISPATCHED AFTER COMMIT — DomainEvents.cs

```
public interface IDomainEvent { DateTimeOffset OccurredOn { get; } }

public sealed record ClaimSubmitted(Guid ClaimId, string PayerCode) : IDomainEvent
{
    public DateTimeOffset OccurredOn { get; } = DateTimeOffset.UtcNow;
}

public abstract class AggregateRoot
{
    private readonly List<IDomainEvent> _events = [];
    public IReadOnlyCollection<IDomainEvent> DomainEvents => _events.AsReadOnly();

    protected void Raise(IDomainEvent e) => _events.Add(e);
    public void ClearEvents() => _events.Clear();
}

public sealed class Claim : AggregateRoot
{
    public void Submit()
    {
        Status = ClaimStatus.Submitted;
        Raise(new ClaimSubmitted(Id, PayerCode));   // subject notifies, knows no observer
    }
}

// Observers register themselves; the aggregate never learns their names.
public sealed class NotifyBillingOnSubmission(IEmailSender email)
    : INotificationHandler<ClaimSubmitted>
{
    public Task Handle(ClaimSubmitted e, CancellationToken ct) =>
        email.SendAsync("billing@example.org", $"Claim {e.ClaimId} submitted", ct);
}
```

WHERE THIS REAPPEARS IN PART 3

Dispatching domain events *after* the database transaction commits is a natural job for the Unit of Work. We will wire exactly this in §3.9.

Template Method

Intent: define the skeleton of an algorithm in a base class and let subclasses override individual steps without changing the overall order.

TEMPLATE METHOD — FIXED IMPORT PIPELINE, PLUGGABLE PARSING — ImportJob.cs

```
public abstract class ImportJob<TRow>
{
    // The template: order is fixed here and nowhere else.
    public async Task<ImportReport> RunAsync(Stream file, CancellationToken ct)
    {
        var rows     = await ParseAsync(file, ct);          // step 1 — subclass
        var valid    = rows.Where(Validate).ToList();       // step 2 — subclass
        var report   = new ImportReport(rows.Count, valid.Count);

        await PersistAsync(valid, ct);                      // step 3 — subclass
        await OnCompletedAsync(report, ct);                 // step 4 — optional hook
        return report;
    }

    protected abstract Task<IReadOnlyList<TRow>> ParseAsync(Stream file, CancellationToken ct);
    protected abstract bool Validate(TRow row);
    protected abstract Task PersistAsync(IReadOnlyList<TRow> rows, CancellationToken ct);

    protected virtual Task OnCompletedAsync(ImportReport report, CancellationToken ct)
        => Task.CompletedTask;   // hook with a sensible default
}
```

1.7 Telling the categories apart — a working rubric

Ask	If the answer is…	Category	Typical smell that led you here
What varies?	"Which concrete class gets instantiated."	**Creational**	`new` scattered through business logic; constructors with eleven arguments.
	"How objects are plugged together."	**Structural**	Copy-pasted wrappers; a class edited every time a cross-cutting concern is added.
	"Which algorithm runs, or who reacts."	**Behavioural**	Growing `switch` / `if` ladders; callers polling for changes.

PART 1 IN FOUR LINES

- **Creational** — controls *instantiation*. Factory, Builder, Singleton, Prototype, Abstract Factory.
- **Structural** — controls *composition*. Adapter, Decorator, Facade, Proxy, Composite, Bridge, Flyweight.
- **Behavioural** — controls *collaboration*. Strategy, Observer, Command, Template Method, Chain of Responsibility, and seven more.
- The Repository Pattern we spend the rest of this handbook on is *not* in the GoF 23 at all — it comes from Martin Fowler's *PoEAA*. Knowing that distinction is itself worth a mark in the quiz.

PART TWO

2 The Repository Pattern — Intent & Structure

> **Objective 2.** Explain the Repository Pattern's intent and structure — what problem it solves, who its participants are, and how it differs from the patterns it is routinely confused with.

2.1 The problem it was invented for

Data access has a habit of leaking. It starts as one `_context.Claims.Where(…)` inside a controller, and ends as query logic duplicated across eleven files, none of which can be unit tested without a database.

Look at the code below. Nothing in it is *wrong*, and you will find code exactly like it in most real ASP.NET Core projects. The problem is what it makes difficult.

THE STARTING POINT — PERSISTENCE CONCERNS EVERYWHERE — ClaimsController.cs

```
[ApiController, Route("api/claims")]
public class ClaimsController(AppDbContext db) : ControllerBase
{
    [HttpGet("outstanding")]
    public async Task<IActionResult> GetOutstanding(string payerCode)
    {
        // Business rule "what counts as outstanding" is expressed as an EF query,
        // in a controller, in the presentation layer.
        var claims = await db.Claims
            .Include(c => c.Lines)
            .Include(c => c.Patient)
            .Where(c => c.PayerCode == payerCode
                     && c.Status != ClaimStatus.Paid
                     && c.Status != ClaimStatus.Voided
                     && c.SubmittedOn < DateTime.UtcNow.AddDays(-30)
                     && !c.IsDeleted)
            .OrderByDescending(c => c.SubmittedOn)
            .ToListAsync();

        return Ok(claims);
    }
}
```

Four costs, and they compound:

Cost	What it looks like six months later
Duplicated rules	"Outstanding" is redefined in the controller, a background job and a report service. A rule change fixes two of the three. The third quietly produces wrong numbers.
Untestable logic	Testing the rule means spinning up a database. Test suites get slow, then get skipped.
Leaked infrastructure	`Include`, `AsNoTracking`, `IsDeleted` and provider quirks are EF Core vocabulary sitting in code that should only speak the domain's.
Hard to change storage	Moving one hot query to Dapper, a read replica or a cache means touching every caller.

Table 2.1 — What unmediated data access costs. Note that none of these costs is "EF Core is bad". They are all consequences of *where* the query lives.

2.2 Intent

> **DEFINITION — FOWLER, PATTERNS OF ENTERPRISE APPLICATION ARCHITECTURE (2002)**
>
> A Repository *"mediates between the domain and data mapping layers using a collection-like interface for accessing domain objects."*
>
> Two words carry the whole pattern: **mediates** — nothing above it talks to the database directly — and **collection-like** — callers should feel like they are working with an in-memory collection of domain objects, not issuing queries.

The intent, restated for a code review:

- **Give the domain an illusion.** Business code asks for objects (`GetOutstandingAsync(payerCode)`) rather than describing how to fetch rows.
- **Centralise query knowledge.** Each persistence rule is written once, in one type, with a name that states the rule.
- **Make persistence substitutable.** Because callers depend on an interface, the implementation can be EF Core, Dapper, an HTTP API or an in-memory fake.
- **Make business logic testable without I/O.** The service layer under test takes an `IClaimRepository`; the test gives it a stub.

> **NOT THE INTENT**
>
> "So we can swap SQL Server for MongoDB one day" is the reason most often given and almost never realised. It is a *possible* consequence, not the intent, and it is the weakest argument you can bring to a design discussion. §4.2 unpacks why.

2.3 Structure and participants

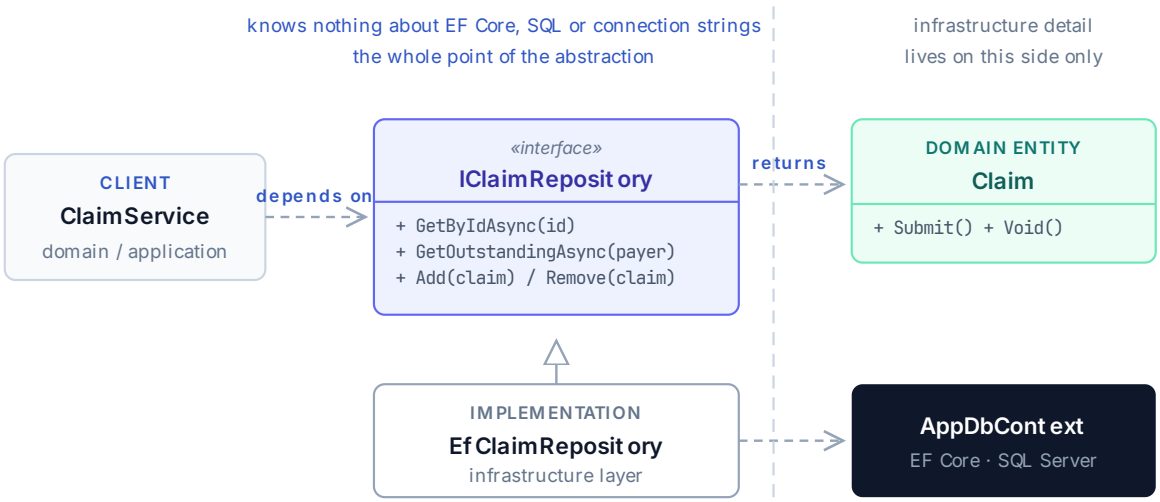


Figure 2.1 — Participants. Client asks for domain objects. **Repository interface** states what can be asked, in domain language, and is owned by the domain side. **Repository implementation** and the **data source** live on the infrastructure side. The dependency arrow crosses the boundary in one direction only — this is the Dependency Inversion Principle in its most common real-world form.

Participant	Lives in	Responsibility
Repository interface `IClaimRepository`	Domain / Application	Declares the questions the domain is allowed to ask, in the domain's own words. Contains no EF Core types.
Concrete repository `EfClaimRepository`	Infrastructure	Translates those questions into queries. Owns `Include`, indexes hints, projections, provider quirks.
Entity / Aggregate `Claim`	Domain	The object handed back and forth. Holds behaviour, not just properties.
Data source `AppDbContext`	Infrastructure	The actual mapper/session. Never referenced above the repository.
Unit of Work `IUnitOfWork`	Both sides	Decides *when* the accumulated changes are committed. Covered in Part 3.

Table 2.2 — Who does what. If an EF Core type appears in the left-hand column's first row, the pattern has already failed.

2.4 The same query, mediated

AFTER — THE RULE HAS A NAME AND ONE HOME — IClaimRepository.cs + EfClaimRepository.cs

```
// ---- Domain layer: the contract, in domain language -------------------------
public interface IClaimRepository
{
    Task<Claim?> GetByIdAsync(Guid id, CancellationToken ct = default);
    Task<IReadOnlyList<Claim>> GetOutstandingAsync(string payerCode, CancellationToken ct = default);
    void Add(Claim claim);
    void Remove(Claim claim);
}

// ---- Infrastructure layer: the only place EF Core appears -------------------
public sealed class EfClaimRepository(AppDbContext db) : IClaimRepository
{
    private static readonly TimeSpan OutstandingAfter = TimeSpan.FromDays(30);

    public Task<Claim?> GetByIdAsync(Guid id, CancellationToken ct = default) =>
        db.Claims.Include(c => c.Lines)
                 .FirstOrDefaultAsync(c => c.Id == id, ct);

    public async Task<IReadOnlyList<Claim>> GetOutstandingAsync(
        string payerCode, CancellationToken ct = default)
    {
        var cutoff = DateTime.UtcNow - OutstandingAfter;

        return await db.Claims
            .AsNoTracking()                      // read-only path: skip change tracking
            .Include(c => c.Lines)
            .Include(c => c.Patient)
            .Where(c => c.PayerCode == payerCode
                     && c.Status != ClaimStatus.Paid
                     && c.Status != ClaimStatus.Voided
                     && c.SubmittedOn < cutoff)
            .OrderByDescending(c => c.SubmittedOn)
            .ToListAsync(ct);
    }

    public void Add(Claim claim)    => db.Claims.Add(claim);
    public void Remove(Claim claim) => db.Claims.Remove(claim);
}
```

```
// ---- Caller: reads as a sentence, testable without a database ---------------
public sealed class ChaseOutstandingClaims(IClaimRepository claims, INotifier notifier)
{
    public async Task RunAsync(string payerCode, CancellationToken ct = default)
    {
        foreach (var claim in await claims.GetOutstandingAsync(payerCode, ct))
            await notifier.SendAsync($"Claim {claim.Number} is overdue.", ct);
    }
}
```

NOTICE WHAT CHANGED AND WHAT DID NOT

The LINQ is identical. The *only* change is which file it lives in — and that single move is what makes the rule nameable, reusable, testable and replaceable. Patterns are usually about location, not cleverness.

2.5 Where the repository sits in a layered architecture

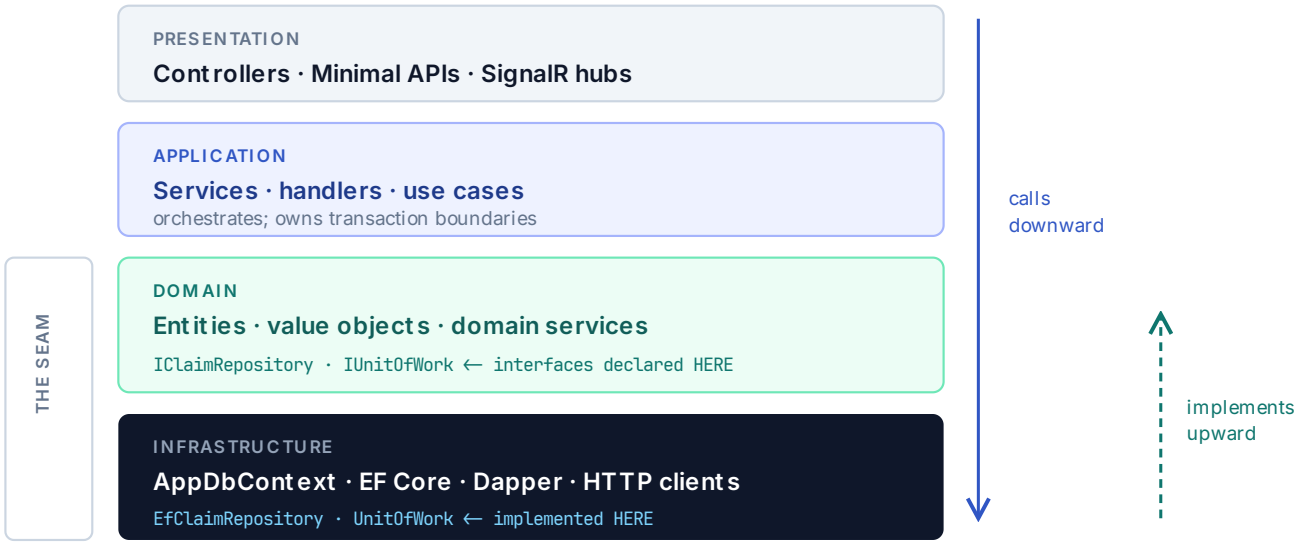


Figure 2.2 — Placement. The interface is declared in the layer that *needs* it (domain/application); the implementation lives in the layer that *knows how* (infrastructure). Calls go down; the implementation dependency goes up. Get this backwards — interface in the infrastructure project — and you have a folder rename, not a pattern.

2.6 Patterns it is confused with

Half of all repository arguments are two people using the same word for different things. This table is worth reading twice.

Pattern	Core idea	How it differs from Repository
Repository	A collection-like façade over persisted *aggregates*.	—
DAO (Data Access Object)	One object per *table*, exposing CRUD for that table.	DAO is table-shaped and speaks SQL/rows; Repository is domain-shaped and speaks objects. A "repository" with one class per table is really a DAO wearing the wrong name.
Data Mapper	Moves data between objects and the database, keeping them independent.	EF Core *is* a Data Mapper. Repository sits *on top of* a data mapper; it does not replace one.

Pattern	Core idea	How it differs from Repository
Active Record	The entity saves itself: `claim.Save()`.	Opposite philosophy — persistence lives inside the entity instead of outside it.
Query Object / Specification	A criteria object that describes a query and can be combined with others.	Complements Repository. Solves the "I need 40 filter combinations" problem that a method-per-query repository cannot (see §3.5).
CQRS read model	Reads bypass the domain entirely and project straight to DTOs.	Often the better answer for reporting screens. Repository stays on the write side.
Unit of Work	Tracks changed objects and commits them together.	A partner, not a rival. Repository decides *what* changes; UoW decides *when* it is written.

Table 2.3 — Neighbouring patterns. All except Active Record can coexist with Repository in one codebase — and frequently should.

2.7 The honest objection: "EF Core is already a repository"

You will hear this in every discussion, and it deserves a straight answer rather than a dismissal, because it is **substantially true**.

THE ARGUMENT FOR

- `DbSet<Claim>` is a collection-like façade over persisted objects — the literal definition of a repository.
- `DbContext` tracks changes and commits them atomically in `SaveChangesAsync` — the literal definition of a Unit of Work.
- Wrapping them in `IRepository<T>` can therefore be a rename, adding a file and removing capability.
- EF Core already has an in-memory and SQLite provider, so "testability" is a weaker argument than it was in 2010.

THE ARGUMENT AGAINST

- `DbSet<T>` exposes *everything*: any caller can write any query, including a badly indexed one, in a controller.
- It offers no place to *name* a persistence rule, so rules get duplicated.
- It is an `IQueryable<T>`, so callers bind to LINQ-to-Entities translation rules — a very leaky abstraction.
- Testing against a different provider is testing a different database: relational behaviour, concurrency and SQL translation all differ.

> **THE RESOLUTION WE WILL USE FOR THE REST OF THIS HANDBOOK**
>
> Do not add a repository to *abstract EF Core*. Add one to **name your domain's persistence rules and constrain what callers may ask**. If your repository's methods are `GetAll`, `GetById`, `Add`, `Update`, `Delete`, you have added a layer and no vocabulary — that is the case Part 4 tells you to delete.

2.8 The leak that decides the design

There is one design decision inside the pattern that matters more than all the others: does the repository return `IQueryable<T>`, or materialised results?

RETURNING IQUERYABLE — THE ABSTRACTION IS FAKE

```
public interface IClaimRepository
{
    IQueryable<Claim> Query();
}

// The caller can now do anything at all:
var page = repo.Query()
    .Include(c => c.Lines)          // EF Core
again
    .Where(c => c.Patient.Name      // N+1 waiting
                .StartsWith(q))
    .Skip(500).Take(50)
    .ToList();                      // sync-over-IO
```

RETURNING RESULTS — THE BOUNDARY HOLDS

```
public interface IClaimRepository
{
    Task<PagedResult<Claim>> SearchAsync(
        ClaimSearch criteria,
        CancellationToken ct = default);
}

// The caller states intent; the repository owns
// paging, includes, indexes and translation.
var page = await repo.SearchAsync(
    new ClaimSearch(PayerCode: "PBM-01",
                    NameStartsWith: q,
                    Page: 11, PageSize: 50), ct);
```

Exposing `IQueryable<T>` gives callers the full power of LINQ *and* the full set of its failure modes: queries that silently evaluate in memory, lazy loading triggered after the context is disposed, and expressions the provider cannot translate. You gain a file and lose the guarantee the pattern exists to provide.

THE RULE

A repository method returns **entities, aggregates, DTOs or a paged result** — never a query. If callers legitimately need many filter combinations, give them a *Specification* object (§3.5), not an `IQueryable`.

PART 2 IN FOUR LINES

- **Intent:** mediate between domain and data mapping using a collection-like interface.
- **Structure:** interface owned by the domain, implementation owned by infrastructure, entity passed between them.
- **Value:** persistence rules get names, one home, and a test seam.
- **Failure mode:** returning `IQueryable<T>`, or having no vocabulary beyond CRUD.

PART THREE

3 Building a Generic Repository with Unit of Work

> **Objective 3.** Implement a generic repository with a Unit of Work pattern — end to end, in ASP.NET Core and EF Core, including specifications, DI lifetimes, transactions and tests.

3.1 What we are building

A generic repository gives every aggregate the same base capabilities without copy-pasting them; a Unit of Work makes one HTTP request equal one database transaction, no matter how many repositories took part in it.

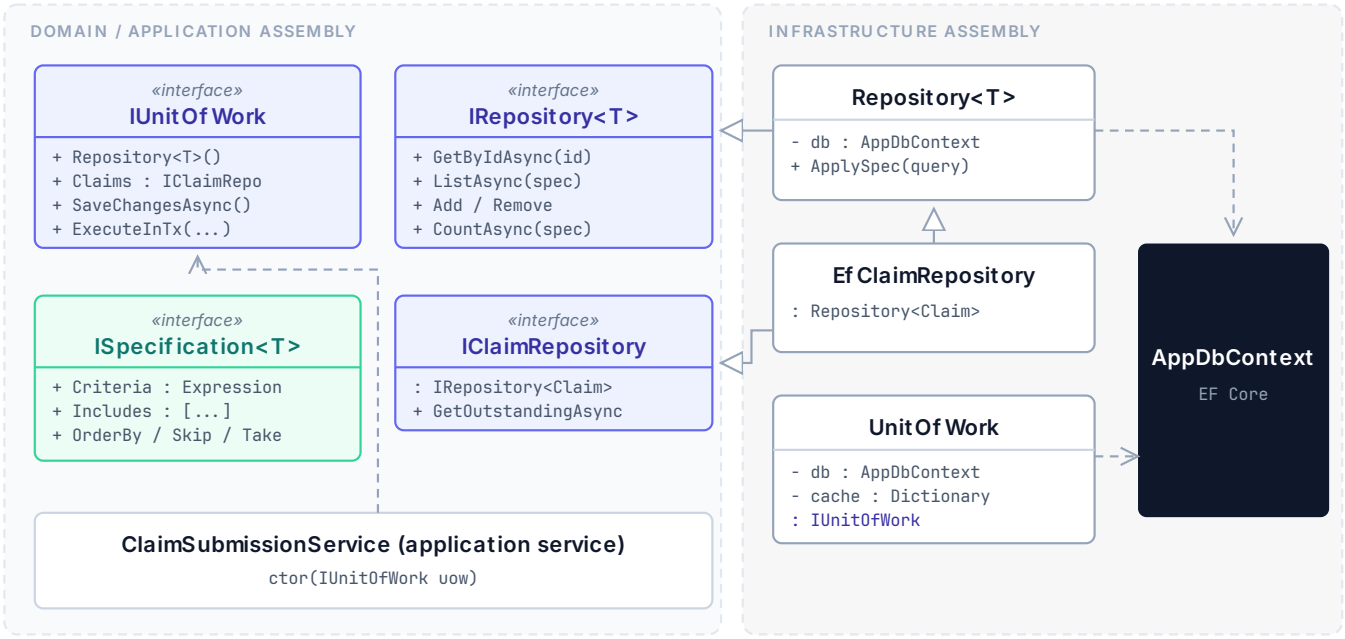


Figure 3.1 — Target design. One shared `Repository<T>` supplies the mechanics; `EfClaimRepository` adds the vocabulary that is specific to claims; `UnitOfWork` owns the single `AppDbContext` that all of them share, which is what makes one commit possible.

> **THE CRITICAL INVARIANT**
>
> Every repository in one request must hold **the same `DbContext` instance**. If two repositories get two contexts, they have two change trackers and two transactions, and "Unit of Work" becomes a name on a class that does not do its job. In ASP.NET Core, `AddDbContext` registers the context as *scoped*, which gives you this for free — provided nothing captures it in a singleton (§1.4).

3.2 Step 1 — entity base and DbContext

DOMAIN FOUNDATION — Domain/Common/Entity.cs

```
public abstract class Entity
{
    public Guid Id { get; protected set; } = Guid.CreateVersion7();
    // Version-7 GUIDs are time-ordered — far kinder to clustered indexes than Guid.NewGuid().
}

public interface ISoftDeletable
{
    bool IsDeleted { get; set; }
```

```
    DateTimeOffset? DeletedOn { get; set; }
}

public interface IAuditable
{
    DateTimeOffset CreatedOn  { get; set; }
    string?        CreatedBy  { get; set; }
    DateTimeOffset? ModifiedOn { get; set; }
    string?        ModifiedBy  { get; set; }
}

public abstract class AggregateRoot : Entity
{
    private readonly List<IDomainEvent> _events = [];
    public IReadOnlyCollection<IDomainEvent> DomainEvents => _events.AsReadOnly();
    protected void Raise(IDomainEvent e) => _events.Add(e);
    public void ClearDomainEvents() => _events.Clear();
}
```

THE CONTEXT — ONE PER REQUEST, OWNED BY THE UNIT OF WORK — Infrastructure/AppDbContext.cs

```
public sealed class AppDbContext(DbContextOptions<AppDbContext> options) : DbContext(options)
{
    public DbSet<Claim>   Claims   => Set<Claim>();
    public DbSet<Patient> Patients => Set<Patient>();

    protected override void OnModelCreating(ModelBuilder b)
    {
        b.ApplyConfigurationsFromAssembly(typeof(AppDbContext).Assembly);

        // Soft delete applied once, globally — repositories never repeat "!IsDeleted".
        foreach (var et in b.Model.GetEntityTypes()
                            .Where(t => typeof(ISoftDeletable).IsAssignableFrom(t.ClrType)))
        {
            var p = Expression.Parameter(et.ClrType, "e");
            var filter = Expression.Lambda(
                Expression.Not(Expression.Property(p, nameof(ISoftDeletable.IsDeleted))), p);
            b.Entity(et.ClrType).HasQueryFilter(filter);
        }

        base.OnModelCreating(b);
    }
}
```

3.3 Step 2 — the repository contract

Keep it deliberately small. Every method you add here is a method every aggregate inherits whether it makes sense for it or not.

GENERIC CONTRACT — NO EF CORE TYPES CROSS THIS LINE — Domain/Abstractions/IRepository.cs

```
public interface IRepository<T> where T : Entity
{
    Task<T?> GetByIdAsync(Guid id, CancellationToken ct = default);

    Task<T?>                  FirstOrDefaultAsync(ISpecification<T> spec, CancellationToken ct = default);
    Task<IReadOnlyList<T>>    ListAsync(ISpecification<T> spec, CancellationToken ct = default);
    Task<IReadOnlyList<TOut>> ListAsync<TOut>(ISpecification<T, TOut> spec, CancellationToken ct = default);

    Task<int>  CountAsync(ISpecification<T> spec, CancellationToken ct = default);
    Task<bool> AnyAsync(ISpecification<T> spec, CancellationToken ct = default);

    void Add(T entity);
    void AddRange(IEnumerable<T> entities);
    void Update(T entity);
    void Remove(T entity);
}
```

NOTICE WHAT IS MISSING

There is no `SaveChangesAsync` and no `GetAllAsync`. Saving belongs to the Unit of Work — a repository that saves makes atomic multi-aggregate operations impossible. `GetAll` belongs nowhere: it is a production incident with a friendly name.

3.4 Step 3 — the Specification, so queries stay outside the repository

Without specifications, a generic repository has exactly two futures: a method per query (`GetOutstandingByPayerOrderedByDateAsync`…) or a leaked `IQueryable`. The Specification pattern is the third option — a first-class object that *describes* a query and can be named, tested and composed.

SPECIFICATION — CRITERIA, INCLUDES, ORDERING AND PAGING AS DATA — Domain/Abstractions/Specification.cs

```
public interface ISpecification<T>
{
    Expression<Func<T, bool>>?                  Criteria       { get; }
    IReadOnlyList<Expression<Func<T, object>>>  Includes       { get; }
    IReadOnlyList<string>                       IncludeStrings { get; }
    Expression<Func<T, object>>?                OrderBy        { get; }
    Expression<Func<T, object>>?                OrderByDesc    { get; }
    int? Skip { get; }
    int? Take { get; }
    bool AsNoTracking     { get; }
    bool IgnoreQueryFilters { get; }
}

// Projection variant: lets a read-only screen avoid materialising whole aggregates.
public interface ISpecification<T, TOut> : ISpecification<T>
{
    Expression<Func<T, TOut>> Selector { get; }
}

public abstract class Specification<T> : ISpecification<T>
{
    protected Specification(Expression<Func<T, bool>>? criteria = null) => Criteria = criteria;

    public Expression<Func<T, bool>>? Criteria { get; private set; }
    public IReadOnlyList<Expression<Func<T, object>>> Includes => _includes;
    public IReadOnlyList<string> IncludeStrings => _includeStrings;
    public Expression<Func<T, object>>? OrderBy     { get; private set; }
```

```
    public Expression<Func<T, object>>? OrderByDesc { get; private set; }
    public int? Skip { get; private set; }
    public int? Take { get; private set; }
    public bool AsNoTracking       { get; private set; }
    public bool IgnoreQueryFilters { get; private set; }

    private readonly List<Expression<Func<T, object>>> _includes = [];
    private readonly List<string> _includeStrings = [];

    protected void AddInclude(Expression<Func<T, object>> include) => _includes.Add(include);
    protected void AddInclude(string includeString)                 => _includeStrings.Add(includeString);
    protected void ApplyOrderBy(Expression<Func<T, object>> e)      => OrderBy = e;
    protected void ApplyOrderByDescending(Expression<Func<T, object>> e) => OrderByDesc = e;
    protected void ApplyPaging(int skip, int take)  { Skip = skip; Take = take; }
    protected void AsReadOnly()      => AsNoTracking = true;
    protected void IncludeDeleted()  => IgnoreQueryFilters = true;

    // Composition — the reason specifications beat a method per query.
    // AndAlso is a small extension that rebinds the two lambdas onto one parameter;
    // Ardalis.Specification ships an equivalent if you would rather not write it.
    protected void And(Expression<Func<T, bool>> extra) =>
        Criteria = Criteria is null ? extra : Criteria.AndAlso(extra);
}
```

A NAMED BUSINESS RULE, REUSABLE AND UNIT-TESTABLE ON ITS OWN — Domain/Claims/OutstandingClaimsSpec.cs

```
public sealed class OutstandingClaimsSpec : Specification<Claim>
{
    public static readonly TimeSpan Threshold = TimeSpan.FromDays(30);

    public OutstandingClaimsSpec(string payerCode, int page = 1, int pageSize = 50)
        : base(c => c.PayerCode == payerCode
                 && c.Status != ClaimStatus.Paid
                 && c.Status != ClaimStatus.Voided
                 && c.SubmittedOn < DateTime.UtcNow - Threshold)
    {
        AddInclude(c => c.Lines);
        AddInclude(c => c.Patient);
        ApplyOrderByDescending(c => c.SubmittedOn);
        ApplyPaging((page - 1) * pageSize, pageSize);
        AsReadOnly();
    }
}

// The rule can now be asserted in a plain unit test, with no database at all:
//   var spec = new OutstandingClaimsSpec("PBM-01");
//   spec.Criteria!.Compile()(paidClaim).Should().BeFalse();
```

3.5 Step 4 — the generic repository implementation

REPOSITORY<T> — THE ONLY FILE THAT KNOWS EF CORE EXISTS — Infrastructure/Repositories/Repository.cs

```
public class Repository<T>(AppDbContext db) : IRepository<T> where T : Entity
{
    protected AppDbContext Db => db;
    protected DbSet<T> Set => db.Set<T>();

    public virtual Task<T?> GetByIdAsync(Guid id, CancellationToken ct = default) =>
        Set.FirstOrDefaultAsync(e => e.Id == id, ct);

    public virtual Task<T?> FirstOrDefaultAsync(ISpecification<T> spec, CancellationToken ct = default) =>
```

```
        Apply(spec).FirstOrDefaultAsync(ct);

    public virtual async Task<IReadOnlyList<T>> ListAsync(
        ISpecification<T> spec, CancellationToken ct = default) =>
        await Apply(spec).ToListAsync(ct);

    public virtual async Task<IReadOnlyList<TOut>> ListAsync<TOut>(
        ISpecification<T, TOut> spec, CancellationToken ct = default) =>
        await Apply(spec).Select(spec.Selector).ToListAsync(ct);

    // Count must ignore paging, or page 2 reports the wrong total.
    public virtual Task<int> CountAsync(ISpecification<T> spec, CancellationToken ct = default) =>
        Apply(spec, forCount: true).CountAsync(ct);

    public virtual Task<bool> AnyAsync(ISpecification<T> spec, CancellationToken ct = default) =>
        Apply(spec, forCount: true).AnyAsync(ct);

    public virtual void Add(T entity)                      => Set.Add(entity);
    public virtual void AddRange(IEnumerable<T> entities)  => Set.AddRange(entities);
    public virtual void Update(T entity)                   => Set.Update(entity);
    public virtual void Remove(T entity)                   => Set.Remove(entity);

    // ---- the translator: specification -> IQueryable, in one place ----------
    protected IQueryable<T> Apply(ISpecification<T> spec, bool forCount = false)
    {
        IQueryable<T> q = Set;

        if (spec.IgnoreQueryFilters) q = q.IgnoreQueryFilters();
        if (spec.AsNoTracking)       q = q.AsNoTracking();
        if (spec.Criteria is not null) q = q.Where(spec.Criteria);

        if (!forCount)
        {
            q = spec.Includes.Aggregate(q, (current, inc) => current.Include(inc));
            q = spec.IncludeStrings.Aggregate(q, (current, inc) => current.Include(inc));

            if (spec.OrderBy     is not null) q = q.OrderBy(spec.OrderBy);
            if (spec.OrderByDesc is not null) q = q.OrderByDescending(spec.OrderByDesc);

            if (spec.Skip is { } s) q = q.Skip(s);
            if (spec.Take is { } t) q = q.Take(t);
        }

        return q;
    }
}
```

WHY `FORCOUNT` MATTERS

A paged screen needs two queries from one specification: the page, and the total. If `CountAsync` applied `Skip` / `Take` it would return at most `pageSize`. Skipping `Include` for counts also removes joins the database does not need — a measurable win on wide aggregates.

3.6 Step 5 — the Unit of Work

CONTRACT — Domain/Abstractions/IUnitOfWork.cs

```
public interface IUnitOfWork : IAsyncDisposable
{
    IClaimRepository   Claims   { get; }     // named repositories for the aggregates you have
    IPatientRepository Patients { get; }

    IRepository<T> Repository<T>() where T : Entity;   // generic escape hatch for the rest

    Task<int> SaveChangesAsync(CancellationToken ct = default);

    Task<TResult> ExecuteInTransactionAsync<TResult>(
        Func<CancellationToken, Task<TResult>> operation, CancellationToken ct = default);
}
```

IMPLEMENTATION — SHARES ONE CONTEXT, CACHES REPOSITORIES, OWNS THE COMMIT — Infrastructure/UnitOfWork.cs

```
public sealed class UnitOfWork(AppDbContext db, IServiceProvider services, IDomainEventDispatcher events)
    : IUnitOfWork
{
    private readonly Dictionary<Type, object> _repositories = [];
    private IDbContextTransaction? _transaction;

    // Specific repositories are resolved from DI so they can have their own dependencies.
    public IClaimRepository   Claims   => Get<IClaimRepository>();
    public IPatientRepository Patients => Get<IPatientRepository>();

    public IRepository<T> Repository<T>() where T : Entity
    {
        if (_repositories.TryGetValue(typeof(T), out var existing))
            return (IRepository<T>)existing;

        var repo = new Repository<T>(db);       // same context => same change tracker
        _repositories[typeof(T)] = repo;
        return repo;
    }

    private TRepo Get<TRepo>() where TRepo : notnull
    {
        if (_repositories.TryGetValue(typeof(TRepo), out var existing))
            return (TRepo)existing;

        var repo = services.GetRequiredService<TRepo>();
        _repositories[typeof(TRepo)] = repo;
        return repo;
    }

    public async Task<int> SaveChangesAsync(CancellationToken ct = default)
    {
        StampAuditFields();
        ConvertDeletesToSoftDeletes();

        var affected = await db.SaveChangesAsync(ct);

        // Observer pattern from §1.6: publish only after the write succeeded.
        await events.DispatchAsync(CollectAndClearDomainEvents(), ct);
        return affected;
    }

    public async Task<TResult> ExecuteInTransactionAsync<TResult>(
```

```
        Func<CancellationToken, Task<TResult>> operation, CancellationToken ct = default)
    {
        // Nested call: join the transaction already in flight rather than opening a second one.
        if (_transaction is not null) return await operation(ct);

        var strategy = db.Database.CreateExecutionStrategy();   // required for retry-on-failure
        return await strategy.ExecuteAsync(async () =>
        {
            await using var tx = await db.Database.BeginTransactionAsync(ct);
            _transaction = tx;
            try
            {
                var result = await operation(ct);
                await tx.CommitAsync(ct);
                return result;
            }
            catch
            {
                await tx.RollbackAsync(ct);
                throw;
            }
            finally { _transaction = null; }
        });
    }

    public ValueTask DisposeAsync() => db.DisposeAsync();

    // ---- cross-cutting write concerns, applied once for every aggregate -----
    private void StampAuditFields()
    {
        var now  = DateTimeOffset.UtcNow;
        var user = services.GetService<ICurrentUser>()?.UserId;

        foreach (var e in db.ChangeTracker.Entries<IAuditable>())
        {
            if (e.State is EntityState.Added)
            { e.Entity.CreatedOn = now;  e.Entity.CreatedBy = user; }
            else if (e.State is EntityState.Modified)
            { e.Entity.ModifiedOn = now; e.Entity.ModifiedBy = user; }
        }
    }

    private void ConvertDeletesToSoftDeletes()
    {
        foreach (var e in db.ChangeTracker.Entries<ISoftDeletable>()
                            .Where(e => e.State == EntityState.Deleted))
        {
            e.State = EntityState.Modified;
            e.Entity.IsDeleted = true;
            e.Entity.DeletedOn = DateTimeOffset.UtcNow;
        }
    }

    private IReadOnlyList<IDomainEvent> CollectAndClearDomainEvents()
    {
        var roots = db.ChangeTracker.Entries<AggregateRoot>()
                      .Where(e => e.Entity.DomainEvents.Count != 0)
                      .Select(e => e.Entity)
                      .ToList();

        var collected = roots.SelectMany(r => r.DomainEvents).ToList();
        roots.ForEach(r => r.ClearDomainEvents());
        return collected;
```

```
    }
}
```

THIS IS WHERE THE PATTERN EARNS ITS KEEP

Audit stamping, soft deletes and domain-event dispatch are written **once** and apply to every aggregate in the system, for every write path, forever. That is a much stronger justification for a Unit of Work than “we might change database one day”.

3.7 Step 6 — the sequence of one request

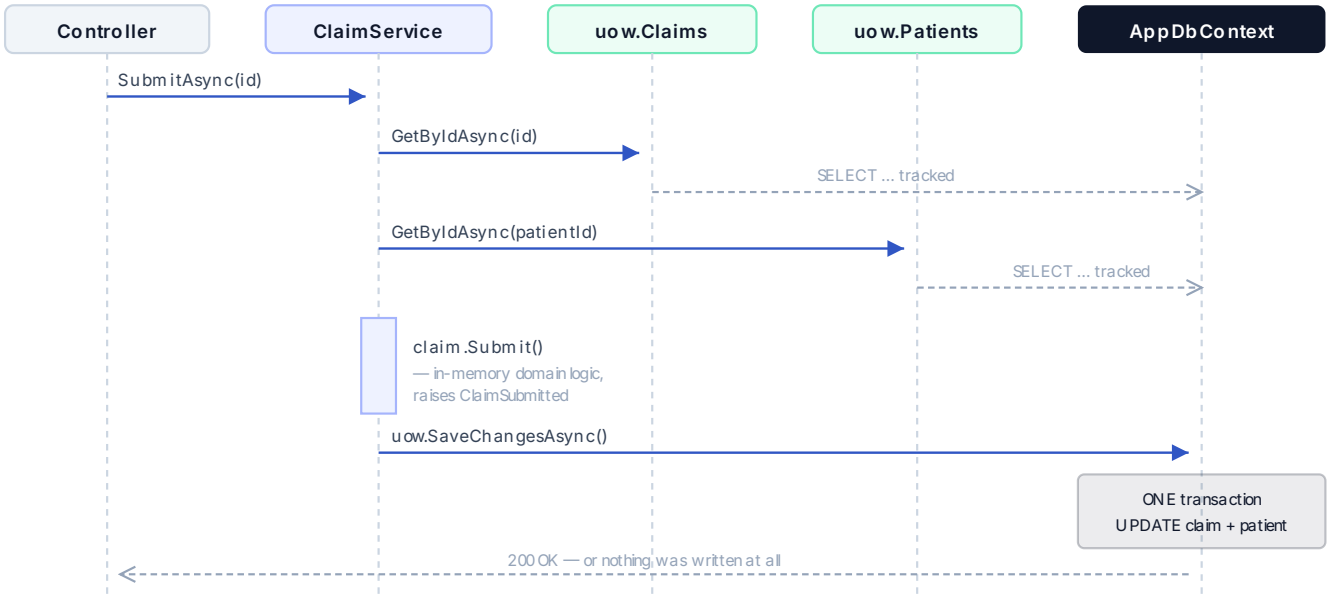


Figure 3.2 — One request, two aggregates, one commit. Both repositories read through the same context, so both modified entities sit in one change tracker. A single `SaveChangesAsync` writes them in one transaction — the property the pattern is named after.

3.8 Step 7 — registration and lifetimes

COMPOSITION ROOT — Program.cs / DependencyInjection.cs

```
public static IServiceCollection AddPersistence(
    this IServiceCollection services, IConfiguration config)
{
    services.AddDbContext<AppDbContext>(o =>
        o.UseSqlServer(config.GetConnectionString("Default"), sql =>
        {
            sql.EnableRetryOnFailure(maxRetryCount: 3);
            sql.CommandTimeout(30);
        }));                                        // scoped by default — exactly what we want

    // Open generic: IRepository<Claim>, IRepository<Patient>, … all satisfied by one registration.
    services.AddScoped(typeof(IRepository<>), typeof(Repository<>));

    // Aggregates that earned their own vocabulary.
    services.AddScoped<IClaimRepository,   EfClaimRepository>();
    services.AddScoped<IPatientRepository, EfPatientRepository>();

    services.AddScoped<IUnitOfWork, UnitOfWork>();
    services.AddScoped<IDomainEventDispatcher, DomainEventDispatcher>();
```

```
    return services;
}
```

Component	Lifetime	Why — and what breaks otherwise
`AppDbContext`	**Scoped**	One change tracker per request. *Singleton* → cross-request data leaks and `DbContext` is not thread-safe. *Transient* → every repository gets its own tracker and the Unit of Work stops working.
`Repository<T>`	**Scoped**	Must match the context's lifetime. A transient repository is harmless only because it still receives the scoped context — keep them aligned to avoid surprises.
`IUnitOfWork`	**Scoped**	It *is* the request. Scoped is the definition of the unit.
Application services	**Scoped**	They depend on `IUnitOfWork`; a singleton service would capture it (§1.4).
`IHostedService` / background workers	**Singleton**	Must *not* inject `IUnitOfWork` directly. Inject `IServiceScopeFactory` and open a scope per work item.

Table 3.1 — Lifetimes. The captive-dependency bug is the single most common way a correct repository implementation fails in production.

THE BACKGROUND-WORKER EXCEPTION, DONE CORRECTLY — Workers/ClaimChaserWorker.cs

```
public sealed class ClaimChaserWorker(IServiceScopeFactory scopeFactory, ILogger<ClaimChaserWorker> log)
    : BackgroundService
{
    protected override async Task ExecuteAsync(CancellationToken ct)
    {
        using var timer = new PeriodicTimer(TimeSpan.FromHours(1));
        while (await timer.WaitForNextTickAsync(ct))
        {
            await using var scope = scopeFactory.CreateAsyncScope();   // fresh scope == fresh unit of work
            var uow = scope.ServiceProvider.GetRequiredService<IUnitOfWork>();
            try
            {
                var overdue = await uow.Claims.ListAsync(new OutstandingClaimsSpec("PBM-01"), ct);
                foreach (var claim in overdue) claim.FlagForChase();
                await uow.SaveChangesAsync(ct);
            }
            catch (Exception ex) { log.LogError(ex, "Claim chase run failed."); }
        }
    }
}
```

3.9 Step 8 — using it, and the aggregate-specific repository

A SPECIFIC REPOSITORY INHERITS THE MECHANICS AND ADDS VOCABULARY — Infrastructure/Repositories/EfClaimRepository.cs

```
public interface IClaimRepository : IRepository<Claim>
{
    Task<Claim?> GetWithLinesAsync(Guid id, CancellationToken ct = default);
    Task<bool>   NumberExistsAsync(string number, CancellationToken ct = default);
    Task<IReadOnlyList<PayerTotal>> TotalsByPayerAsync(DateOnly from, DateOnly to, CancellationToken ct =
default);
```

```
}

public sealed class EfClaimRepository(AppDbContext db) : Repository<Claim>(db), IClaimRepository
{
    public Task<Claim?> GetWithLinesAsync(Guid id, CancellationToken ct = default) =>
        Set.Include(c => c.Lines).FirstOrDefaultAsync(c => c.Id == id, ct);

    public Task<bool> NumberExistsAsync(string number, CancellationToken ct = default) =>
        Set.AnyAsync(c => c.Number == number, ct);

    // Aggregations do not belong in a generic contract — put them where they are understood.
    public async Task<IReadOnlyList<PayerTotal>> TotalsByPayerAsync(
        DateOnly from, DateOnly to, CancellationToken ct = default) =>
        await Set.AsNoTracking()
                 .Where(c => c.ServiceDate >= from && c.ServiceDate <= to)
                 .GroupBy(c => c.PayerCode)
                 .Select(g => new PayerTotal(g.Key, g.Count(), g.Sum(c => c.TotalAmount)))
                 .ToListAsync(ct);
}
```

THE APPLICATION SERVICE — THE PAYOFF — Application/Claims/ClaimSubmissionService.cs

```
public sealed class ClaimSubmissionService(IUnitOfWork uow, IPricingEngine pricing)
{
    public async Task<Result<Guid>> SubmitAsync(SubmitClaimCommand cmd, CancellationToken ct = default)
    {
        if (await uow.Claims.NumberExistsAsync(cmd.Number, ct))
            return Result<Guid>.Conflict($"Claim number '{cmd.Number}' already exists.");

        var patient = await uow.Patients.GetByIdAsync(cmd.PatientId, ct);
        if (patient is null)
            return Result<Guid>.NotFound($"Patient {cmd.PatientId} was not found.");

        // Two aggregates change; one transaction covers both.
        return await uow.ExecuteInTransactionAsync(async token =>
        {
            var claim = Claim.Create(patient, cmd.Number, cmd.ServiceDate, cmd.PayerCode);
            foreach (var line in cmd.Lines)
                claim.AddLine(line.Code, line.Quantity, pricing.UnitPrice(line.Code));

            claim.Submit();                       // raises ClaimSubmitted
            patient.RecordSubmission(claim.Id);   // second aggregate, same transaction

            uow.Claims.Add(claim);
            await uow.SaveChangesAsync(token);

            return Result<Guid>.Success(claim.Id);
        }, ct);
    }
}
```

READ THE SERVICE AGAIN

There is no `DbContext`, no `Include`, no `SaveChanges` on a repository, no connection string and no SQL. Every line is either a business rule or a transaction boundary. That readability — not database portability — is the return on the indirection.

3.10 Step 9 — concurrency

Two users editing the same claim is not an edge case, it is Tuesday. Optimistic concurrency belongs in the repository/UoW layer because that is where `SaveChangesAsync` lives.

OPTIMISTIC CONCURRENCY WITH A ROWVERSION TOKEN — Infrastructure + Application

```
// 1. Mapping — SQL Server maintains the token for you.
public sealed class ClaimConfiguration : IEntityTypeConfiguration<Claim>
{
    public void Configure(EntityTypeBuilder<Claim> b)
    {
        b.HasKey(c => c.Id);
        b.Property(c => c.Number).HasMaxLength(32).IsRequired();
        b.HasIndex(c => c.Number).IsUnique();
        b.Property(c => c.RowVersion).IsRowVersion();       // <- the concurrency token
        b.OwnsMany(c => c.Lines, l => l.ToTable("ClaimLines"));
    }
}

// 2. Handling — translate the infrastructure failure into a domain-meaningful result.
try
{
    await uow.SaveChangesAsync(ct);
}
catch (DbUpdateConcurrencyException ex)
{
    var entry    = ex.Entries.Single();
    var database = await entry.GetDatabaseValuesAsync(ct);

    if (database is null)
        return Result.Gone("The claim was deleted by another user.");

    return Result.Conflict(
        "The claim was modified by another user. Reload and try again.",
        currentVersion: database.GetValue<byte[]>(nameof(Claim.RowVersion)));
}
```

3.11 Step 10 — testing

Two kinds of test, two different tools. Confusing them is why people conclude “repositories don't help with testing”.

UNIT SERVICE LOGIC

Substitute `IUnitOfWork` and the repositories. No database, runs in milliseconds, asserts *business rules*.

INTEGRATION QUERY CORRECTNESS

Use a real relational provider — SQLite in-memory or Testcontainers — to assert that specifications *translate* and return the right rows.

UNIT TEST — THE RULE, NOT THE DATABASE — Tests/ClaimSubmissionServiceTests.cs

```
[Fact]
public async Task SubmitAsync_returns_conflict_when_claim_number_already_exists()
{
    var claims = Substitute.For<IClaimRepository>();
    claims.NumberExistsAsync("CLM-001", Arg.Any<CancellationToken>()).Returns(true);

    var uow = Substitute.For<IUnitOfWork>();
    uow.Claims.Returns(claims);

    var sut = new ClaimSubmissionService(uow, Substitute.For<IPricingEngine>());

    var result = await sut.SubmitAsync(new SubmitClaimCommand(Number: "CLM-001", /* … */));

    result.Status.Should().Be(ResultStatus.Conflict);
    await uow.DidNotReceive().SaveChangesAsync(Arg.Any<CancellationToken>());
}
```

INTEGRATION TEST — SQLITE IN-MEMORY, A REAL RELATIONAL PROVIDER — Tests/OutstandingClaimsSpecTests.cs

```
public sealed class SqliteFixture : IAsyncLifetime
{
    private SqliteConnection _connection = null!;
    public AppDbContext Db { get; private set; } = null!;

    public async Task InitializeAsync()
    {
        _connection = new SqliteConnection("DataSource=:memory:");
        await _connection.OpenAsync();                // keep open or the database vanishes

        Db = new AppDbContext(new DbContextOptionsBuilder<AppDbContext>()
            .UseSqlite(_connection).Options);
        await Db.Database.EnsureCreatedAsync();
    }

    public async Task DisposeAsync()
    {
        await Db.DisposeAsync();
        await _connection.DisposeAsync();
    }
}

public sealed class OutstandingClaimsSpecTests(SqliteFixture fx) : IClassFixture<SqliteFixture>
{
    [Fact]
    public async Task Excludes_paid_claims_and_claims_newer_than_thirty_days()
    {
        fx.Db.Claims.AddRange(
            Fake.Claim("PBM-01", ClaimStatus.Submitted, submittedDaysAgo: 45),   // expected
            Fake.Claim("PBM-01", ClaimStatus.Paid,      submittedDaysAgo: 45),   // paid
            Fake.Claim("PBM-01", ClaimStatus.Submitted, submittedDaysAgo: 5),    // too recent
            Fake.Claim("PBM-02", ClaimStatus.Submitted, submittedDaysAgo: 45));  // other payer
        await fx.Db.SaveChangesAsync();

        var repo   = new EfClaimRepository(fx.Db);
        var result = await repo.ListAsync(new OutstandingClaimsSpec("PBM-01"));

        result.Should().ContainSingle();
    }
}
```

DO NOT TEST QUERIES AGAINST THE INMEMORY PROVIDER

EF Core's `Microsoft.EntityFrameworkCore.InMemory` provider is not a relational database: it has no joins, no constraints, no transactions and different null/ordering semantics. Queries that pass there routinely fail against SQL Server. Use SQLite in-memory for speed, or Testcontainers with the real engine for confidence.

3.12 Bugs this design is built to prevent

Bug	Symptom in production	Guard in the code above
Partial writes	Claim saved, patient counter not — data disagrees with itself.	One `SaveChangesAsync` per operation; `ExecuteInTransactionAsync` for multi-step work.
Captive dependency	`ObjectDisposedException`, or one context shared across requests.	Scoped registrations; `IServiceScopeFactory` in background workers (§3.8).
Wrong page totals	"Showing 1–50 of 50" on every page.	`forCount` skips `Skip` / `Take` in `Apply`.
N+1 queries	One list screen issues 200 SELECTs; p95 latency collapses.	Includes declared on the specification, not left to lazy loading.
Lost concurrency updates	Last writer silently wins; an edit disappears.	`IsRowVersion()` plus explicit `DbUpdateConcurrencyException` handling.
Events fired on rollback	Email says "claim submitted" for a claim that was never saved.	Dispatch happens *after* `SaveChangesAsync` returns.
Transient SQL failures	Intermittent 500s under load or during failover.	`EnableRetryOnFailure` + `CreateExecutionStrategy` around the transaction.
Soft-deleted rows resurface	Deleted records appear in one screen but not another.	Global query filter in `OnModelCreating`, opt-out only via the specification.

Table 3.2 — Failure modes and their guards. Each row is a bug the authors of this pattern met before you did.

PART 3 IN FOUR LINES

- `Repository<T>` supplies mechanics; specific repositories supply vocabulary.
- Specifications keep query knowledge out of callers without leaking `IQueryable`.
- `UnitOfWork` owns the context, the commit, and every cross-cutting write concern.
- Scoped lifetimes everywhere; scope-per-work-item in background services.

PART FOUR

4 Judgement — Value vs. Unnecessary Layers

> **Objective 4.** Identify when the Repository Pattern adds value and when it adds unnecessary layers — with a decision procedure you can defend in a code review.

4.1 What the pattern actually buys you

Every one of these is a benefit you can point at in the code. If you cannot point at it, you are not getting it.

Benefit	Evidence it is real in your codebase	Evidence it is imaginary
Named persistence rules	Methods and specifications carry domain names: `GetOutstandingAsync`, `OutstandingClaimsSpec`.	Methods are `GetAll`, `Find`, `Where`.
One home per rule	Changing "outstanding = 30 days" is a one-line edit in one file.	The same predicate appears in three files.
Fast business-logic tests	Service tests run with substitutes and no database, in milliseconds.	Every test needs a database, or there are no service tests.
Cross-cutting write concerns	Audit, soft delete and event dispatch live once, in the Unit of Work.	Each service stamps `ModifiedOn` by hand.
Atomic multi-aggregate operations	One `SaveChangesAsync` per use case; transactions are explicit.	Repositories call `SaveChanges` themselves.
A place to add behaviour	Caching, retry and logging arrive as decorators (§1.5), touching nothing else.	Caching is `if (_cache.TryGet…)` pasted into controllers.
A constrained surface	A junior cannot write an unindexed cross-join from a controller, because there is no `IQueryable` to write it on.	The repository returns `IQueryable<T>`.

Table 4.1 — Benefits and their proof. Use the right-hand column in code review. "We have repositories" is not the same claim as "we get the benefits of repositories".

4.2 What it does not buy you

MYTH **"WE CAN SWAP THE DATABASE"**

The interface hides the query API, not the storage model. Moving from SQL Server to a document store changes your aggregate boundaries, transactional guarantees, id strategy and consistency model. The repository is the smallest part of that migration, and almost nobody ever does it.

MYTH **"IT MAKES THE CODE TESTABLE"**

It makes *callers* testable. The repository itself still needs integration tests against a real provider — and a mocked repository that returns `List<T>` proves nothing about whether your LINQ translates.

MYTH **"IT IS REQUIRED BY CLEAN ARCHITECTURE"**

What Clean/Onion architecture requires is that the domain not depend on infrastructure. A repository is one way to achieve that. So is a narrow application-level interface, or a CQRS handler that owns its own query.

MYTH **"IT IMPROVES PERFORMANCE"**

It is neutral at best. A generic repository that always `Include`s everything, or always materialises whole aggregates for a read screen, makes things measurably worse than a targeted projection.

4.3 The anti-pattern gallery

If you find these in a pull request — including your own — the layer is costing more than it returns.

1 · THE PASS-THROUGH

```
// Adds a file, a mock and an interface.
// Adds no vocabulary, no rule, no guard.
public class ClaimRepository(AppDbContext db)
{
    public IQueryable<Claim> GetAll()
        => db.Claims;

    public void Save() => db.SaveChanges();
}
```

2 · THE REPOSITORY THAT COMMITS

```
public async Task AddAsync(Claim c)
{
    db.Claims.Add(c);
    await db.SaveChangesAsync();   // <-- here
}

// Two repositories in one use case now
// mean two transactions. Atomicity is
// gone, and no caller can restore it.
```

3 · THE GOD REPOSITORY

```
public interface IClaimRepository
{
    Task<...> GetByPayerAndStatusAndDate…
    Task<...> GetByPatientOrderedByDate…
    Task<...> GetByPayerPagedIncludingLines…
    // 47 more methods, each used once
}
// This is the problem specifications
// (§3.4) exist to solve.
```

4 · REPOSITORY PER TABLE

```
IClaimRepository
IClaimLineRepository     // not an aggregate
IClaimLineNoteRepository // not an aggregate

// Claim lines have no life outside a
// Claim. One repository per AGGREGATE
// ROOT — not one per table.
```

5 · LEAKED EF CORE IN THE CONTRACT

```
public interface IClaimRepository
{
    IIncludableQueryable<Claim, Patient>
        WithPatient();

    Task<List<Claim>> FindAsync(
        Expression<Func<Claim,bool>> predicate);
}
// The domain project now references
// EF Core. The boundary is decoration.
```

6 · THE DOUBLE UNIT OF WORK

```
public class UnitOfWork
{
    public UnitOfWork() =>
        _db = new AppDbContext();  // new!

    // A context created here is not the
    // scoped one the repositories got.
    // Two trackers, two transactions,
    // silent data loss.
}
```

4.4 The decision procedure

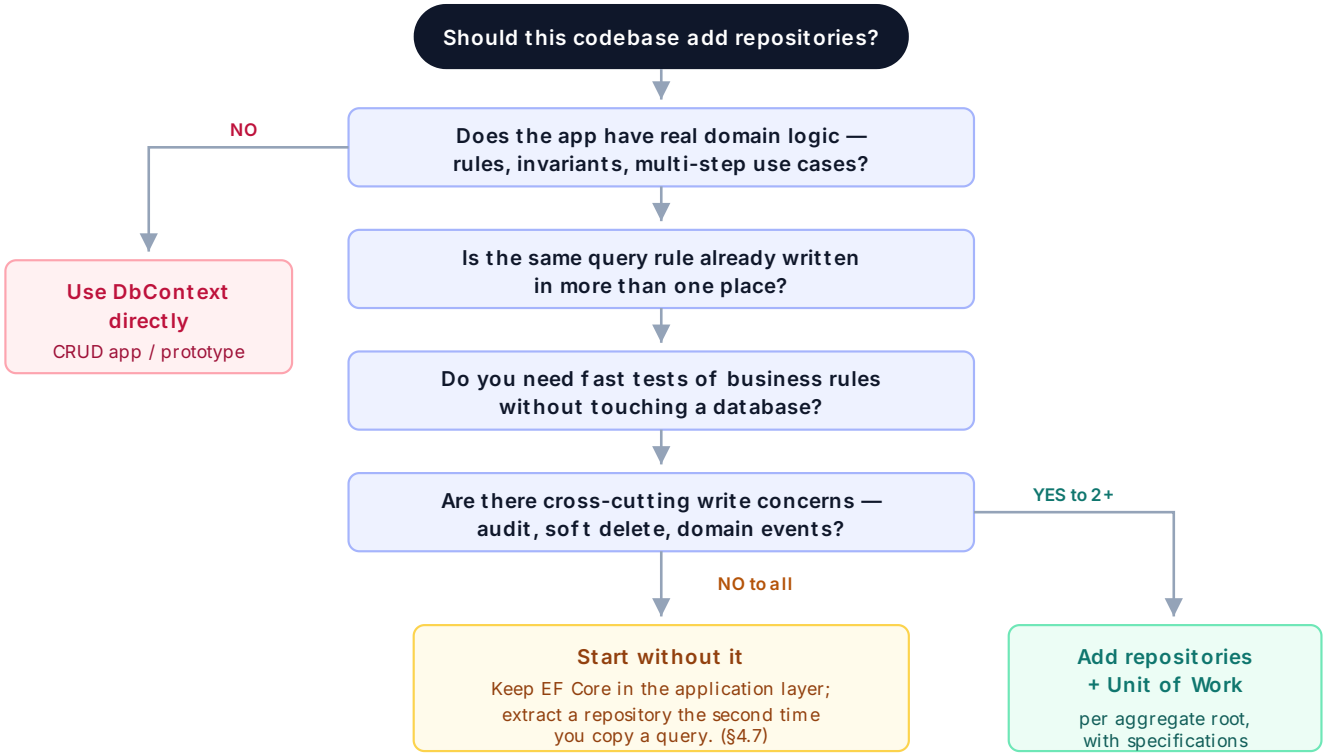


Figure 4.1 — When to introduce the layer. The questions are deliberately about *observed facts in your codebase today*, not about architecture you hope to have. Two or more "yes" answers is the threshold used throughout this handbook.

4.5 Comparing the realistic options

Repository-or-not is a false choice; there are at least five viable data-access designs. Rate your situation against the criteria that matter to you, not against fashion.

Approach	Setup cost	Rules get a name	Fast unit tests	Read performance	Constrains callers	Best fit
DbContext directly no abstraction	Very low	No	No	Full control	No	CRUD services, internal tools, prototypes, small teams.
Specific repositories one per aggregate, no generics	Low	Strong	Strong	Full control	Strong	Most business applications. The default recommendation.
Generic repo + UoW + specs Part 3 of this handbook	Medium	Strong	Strong	Good	Strong	Many aggregates with shared write concerns; large teams.
CQRS handlers write model + projections	Medium	Strong	Good	Excellent	Good	Read-heavy systems; reporting screens; separate scaling.
Generic repo, no specs IRepository<T> with CRUD only	Low	No	Partial	Poor	Partial	**Nothing.** This is the configuration critics mean when they say the pattern is useless.

Table 4.2 — Approaches scored against what teams actually care about. Ratings are editorial judgements for a typical line-of-business ASP.NET Core application, not measurements. Shading encodes the same ordinal scale as the labels, so the table reads correctly in greyscale.

4.6 A checklist you can run in a pull request

GREEN FLAGS — THE LAYER IS EARNING ITS PLACE

- [ ] Repository methods and specs are named after *business* concepts.
- [ ] There is exactly one repository per aggregate root.
- [ ] No `IQueryable`, `Expression<Func<…>>` or EF Core type in the interface.
- [ ] `SaveChangesAsync` exists only on the Unit of Work.
- [ ] Service tests run with no database and finish in milliseconds.
- [ ] Query correctness is covered by integration tests on a relational provider.
- [ ] Cross-cutting write concerns appear exactly once.

RED FLAGS — DELETE THE LAYER, OR FIX IT

- [ ] Every method is `GetAll` / `GetById` / `Add` / `Update` / `Delete`.
- [ ] The interface has one implementation and will only ever have one *and* adds no vocabulary.
- [ ] Callers still write `Include` and `Where` after calling it.
- [ ] Repositories call `SaveChanges` themselves.
- [ ] There is a repository for a child entity that has no independent life.
- [ ] The only stated justification is “we might change database”.
- [ ] Adding one screen means editing four files that contain no logic.

THE TEST TO APPLY WHEN YOU ARE UNSURE

Read the interface out loud with the technology name removed. If it still describes something your business does — *"get the claims this payer has not settled in thirty days"* — keep it. If it only describes something a database does — *"get all, then filter"* — you have written a layer, not an abstraction.

4.7 Getting from here to there

You almost never need a big-bang refactor. The path below adds value at every step and can be stopped at any point.

1. **Stop the bleeding.** Forbid new EF Core queries in controllers. Move existing ones down into application services, unchanged. Cheap, mechanical, immediately better.
2. **Extract on the second copy.** The first time a query is written, leave it. The second time the same predicate appears, pull it into a named specification or repository method. Duplication — not prophecy — is the trigger.
3. **Introduce the Unit of Work when atomicity breaks.** The first bug where two aggregates half-saved is the justification, and it is a much easier conversation than a speculative one.
4. **Add the generic base only when there are three or more repositories** repeating the same mechanics. Before that, the generics cost more than the duplication.
5. **Split reads off when a screen hurts.** Reporting and list screens graduate to projections or a CQRS query handler; the repository stays on the write side, where aggregates matter.
6. **Delete what stopped paying.** A repository that has become a pass-through after a redesign should be removed. Patterns are reversible; treating them as permanent is how codebases calcify.

PART 4 IN FOUR LINES

- The pattern pays for **named rules, one home per rule, fast tests, shared write concerns and a constrained surface**.
- It does not pay for database portability, and saying so weakens your case.
- A CRUD-only `IRepository<T>` over EF Core is the configuration to avoid — it is the critics' exhibit A.
- Introduce it on evidence (duplication, a real atomicity bug), not on prophecy — and remove it when the evidence goes away.

APPENDIX A

A Lab Exercises

> Work through these during or after the session. Each exercise maps to one learning objective and states how you know you are done.

A.1 Warm-up — classify (Objective 1)

For each situation, name the category *and* the pattern you would reach for. Write one sentence on what would change if the pattern were absent.

#	Situation	Your answer
1	A report exporter must support CSV, XLSX and PDF, chosen at run time from a dropdown.	
2	A legacy payments SDK exposes `DoPay(string xml)`; your code expects `IPaymentGateway`.	
3	Every outbound HTTP call should be retried three times and logged, without editing any client.	
4	Building a `Prescription` requires eleven optional fields and three validation rules.	
5	When a claim is approved, four unrelated modules must react, and more will be added.	
6	Import jobs all follow parse → validate → persist → notify, but each parses differently.	
7	Your service needs a different `ITenantConnection` depending on the request's tenant header.	
8	A checkout screen calls seven subsystems; the caller should make one call.	

Answer key. 1 Strategy (behavioural) · 2 Adapter (structural) · 3 Decorator (structural) · 4 Builder (creational) · 5 Observer (behavioural) · 6 Template Method (behavioural) · 7 Factory Method / Abstract Factory (creational) · 8 Facade (structural).

A.2 Core lab — build it (Objectives 2 & 3)

Start from any ASP.NET Core Web API with EF Core and two related entities of your choice (`Order` / `OrderLine` works if you have nothing to hand).

1. **Baseline.** Write one endpoint that queries the context directly in the controller, with at least three predicates. Commit it. This is your "before".
2. **Extract the contract.** Create `IOrderRepository` in the domain project with one method named after the business rule, not the query. The domain project must not reference EF Core — check the `.csproj`.
3. **Implement.** Add `EfOrderRepository` in the infrastructure project. Move the LINQ unchanged. Register it as scoped.
4. **Generalise.** Add `IRepository<T>` and `Repository<T>`; make `EfOrderRepository` inherit it. Register the open generic.

5. **Specify.** Add `ISpecification<T>`, `Specification<T>` and the `Apply` translator. Express your rule as a named specification with includes, ordering and paging.
6. **Unify.** Add `IUnitOfWork` / `UnitOfWork`. Remove every `SaveChangesAsync` from the repositories. Write a use case that modifies **two** aggregates and commits once.
7. **Cross-cut.** Implement audit stamping *or* soft delete inside `SaveChangesAsync`. Verify it applies to an entity you never touched.
8. **Test.** One unit test of the service with a substituted `IUnitOfWork`; one integration test of the specification against SQLite in-memory.

ACCEPTANCE CRITERIA

- [ ] The domain project has no reference to `Microsoft.EntityFrameworkCore`.
- [ ] No repository exposes `IQueryable`, `DbSet` or `Expression`.
- [ ] `SaveChangesAsync` appears on exactly one type.
- [ ] The two-aggregate use case leaves the database unchanged when the second step throws.
- [ ] The unit test suite runs with no database available.
- [ ] `CountAsync` on a paged specification returns the full total, not the page size.

A.3 Judgement drill — argue it (Objective 4)

Split into pairs. One person argues *for* introducing repositories in each scenario, the other *against*. Swap sides after each. Five minutes each; the goal is to find the strongest argument on both sides, not to win.

#	Scenario	Strongest counter-argument
1	An internal admin tool: 12 tables, 3 developers, pure CRUD, no domain rules.	
2	A billing engine: 40 aggregates, audit obligations, 14 developers across 3 teams.	
3	A reporting API: 60 read-only endpoints, heavy aggregation, no writes at all.	
4	A green-field product where the team has never used the pattern before.	
5	An existing codebase with `IRepository<T>` that only exposes CRUD.	

Discussion notes. 1 → §4.4 says no. 2 → yes, with specifications. 3 → CQRS projections beat repositories. 4 → start specific, not generic (§4.7 step 4). 5 → either add vocabulary or delete the layer (§4.6).

APPENDIX B

B Quick Reference

B.1 Repository & Unit of Work cheat sheet

DO

- One repository per **aggregate root**.
- Name methods after business questions.
- Return entities, DTOs or paged results.
- Put `SaveChangesAsync` on the Unit of Work only.
- Register everything **scoped**.
- Use `AsNoTracking` on read-only paths.
- Declare includes on the specification.
- Dispatch domain events after commit.
- Integration-test queries on a relational provider.

DON'T

- Return `IQueryable<T>` or `Expression`.
- Create a repository per table.
- Call `SaveChanges` inside a repository.
- Inject `IUnitOfWork` into a singleton.
- Add `GetAllAsync`.
- Use the EF Core InMemory provider to test queries.
- Instantiate `AppDbContext` with `new`.
- Justify the layer with “we might change database”.
- Keep a pass-through repository out of politeness.

B.2 Minimal skeleton to copy

THE WHOLE PATTERN IN 30 LINES — skeleton.cs

```
// Domain
public interface IRepository<T> where T : Entity
{
    Task<T?> GetByIdAsync(Guid id, CancellationToken ct = default);
    Task<IReadOnlyList<T>> ListAsync(ISpecification<T> spec, CancellationToken ct = default);
    void Add(T e);  void Update(T e);  void Remove(T e);
}

public interface IUnitOfWork : IAsyncDisposable
{
    IRepository<T> Repository<T>() where T : Entity;
    Task<int> SaveChangesAsync(CancellationToken ct = default);
}

// Infrastructure
public class Repository<T>(AppDbContext db) : IRepository<T> where T : Entity
{
    protected DbSet<T> Set => db.Set<T>();
    public Task<T?> GetByIdAsync(Guid id, CancellationToken ct = default) =>
        Set.FirstOrDefaultAsync(e => e.Id == id, ct);
    public async Task<IReadOnlyList<T>> ListAsync(ISpecification<T> s, CancellationToken ct = default) =>
        await SpecificationEvaluator.Apply(Set, s).ToListAsync(ct);
    public void Add(T e) => Set.Add(e);
    public void Update(T e) => Set.Update(e);
    public void Remove(T e) => Set.Remove(e);
}

// Composition root
services.AddDbContext<AppDbContext>(o => o.UseSqlServer(cs));   // scoped
```

```
services.AddScoped(typeof(IRepository<>), typeof(Repository<>));
services.AddScoped<IUnitOfWork, UnitOfWork>();
```

B.3 Glossary

Aggregate
A cluster of entities and value objects treated as one unit for data changes, with a single entry point.

Aggregate root
The entry point entity of an aggregate. The *only* thing that gets a repository.

Change tracker
EF Core's record of which loaded entities were modified. Lives on the `DbContext`; the reason all repositories must share one.

Captive dependency
A short-lived service held alive by a longer-lived one — e.g. a scoped `DbContext` inside a singleton.

Data Mapper
A layer that moves data between objects and the database while keeping them independent. EF Core is one.

Dependency Inversion
High-level modules depend on abstractions; the low-level module supplies the implementation. The reason the interface lives in the domain project.

Domain event
A record that something business-significant happened, raised by an aggregate and dispatched after commit.

Execution strategy
EF Core's retry wrapper. Required around manual transactions when `EnableRetryOnFailure` is on.

GoF
"Gang of Four" — the four authors of *Design Patterns* (1994), and by extension the 23 patterns it catalogues.

Global query filter
A predicate EF Core appends to every query for an entity type — how soft delete is applied in one place.

Optimistic concurrency
Detecting conflicting edits at save time using a version token, rather than locking rows.

PoEAA
Martin Fowler, *Patterns of Enterprise Application Architecture* (2002) — the source of Repository, Unit of Work, Data Mapper and Active Record.

Projection
Selecting only the columns a screen needs, into a DTO, instead of materialising whole aggregates.

Specification
An object that encapsulates query criteria so a rule can be named, composed and tested independently of the repository.

Unit of Work
An object that tracks everything changed during a business transaction and commits it as one atomic write.

B.4 Further reading

Source	Why it is worth your time
Gamma, Helm, Johnson, Vlissides — *Design Patterns* (1994)	The original catalogue. Read the Intent and Consequences sections; skip the C++.
Martin Fowler — *Patterns of Enterprise Application Architecture* (2002)	Where Repository, Unit of Work and Data Mapper are actually defined. Short entries, freely summarised at martinfowler.com/eaaCatalog.
Eric Evans — *Domain-Driven Design* (2003)	Aggregates and aggregate roots — the concept that tells you how many repositories to write.
Vaughn Vernon — *Implementing Domain-Driven Design* (2013)	Practical aggregate design rules, including "one transaction per aggregate" and when to break it.
Microsoft Learn — EF Core documentation	Change tracking, query filters, execution strategies, concurrency tokens and testing guidance, kept current per release.
Microsoft Learn — *.NET Microservices: Architecture for Containerized .NET Applications*	Free e-book with a worked repository + Unit of Work implementation, and a candid section on when not to use it.
Ardalis.Specification (open source)	A production-grade version of the specification code in §3.4, if you would rather adopt than build.

ONE SENTENCE TO TAKE AWAY

Patterns are named trade-offs — reach for one when you can name the change it makes cheap, and delete it when you can't.

Design Patterns & the Repository Pattern — Trainee Handbook. Prepared by **Yazan Faroun** using Claude.